

Git trong VSCode

Cẩm nang dành cho người quen dùng Git trong Visual Studio

Conflict • Diff / Lịch sử • Push / Pull GitHub

Dự án: `train` | Ngày: 15/07/2026

Nội dung

- Phần 1 — Giải quyết conflict khi merge (khác Visual Studio khá nhiều)
- Phần 2 — Xem diff & lịch sử của một file cụ thể
- Phần 3 — Thiết lập Push / Pull với GitHub
- Phần 4 — Xóa branch cũ & tạo branch mới

Ghi nhớ nhanh — 3 phím tắt sống còn:

`Ctrl+Shift+G`

mở Source Control

· `Ctrl+Shift+P`

gõ "Git:" để tìm mọi lệnh

· `Ctrl+``

mở Terminal

Phần 1 — Giải quyết Conflict khi Merge

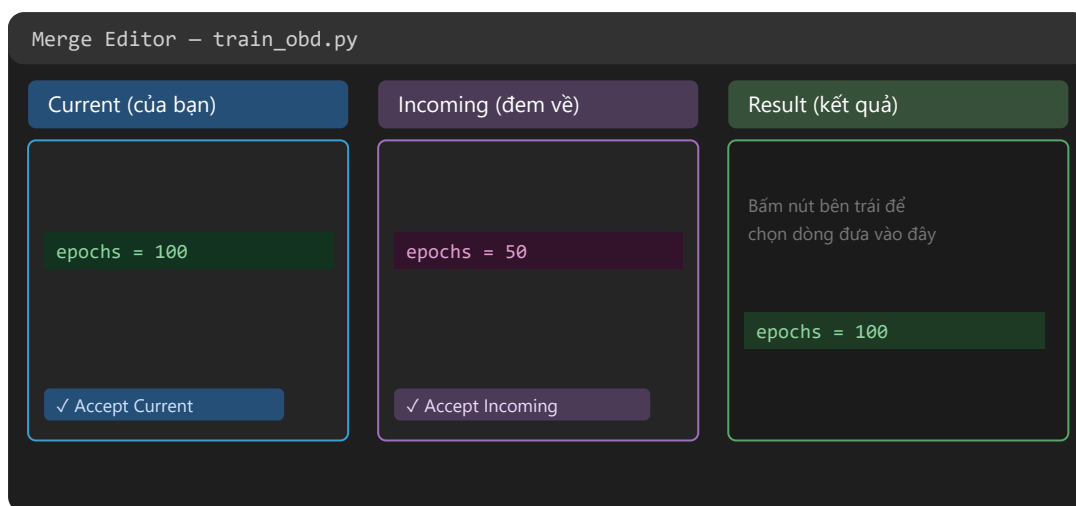
Khi hai nhánh cùng sửa **một dòng** của một file, Git không tự quyết được → sinh ra **conflict**. Trong Visual Studio bạn quen với cửa sổ *Merge Editor* 3 khung; VSCode cũng có Merge Editor nhưng cách vào và thao tác khác.

Chuyện gì xảy ra khi có conflict?

Sau khi chạy `git merge` (hoặc pull) mà đưng độ, VSCode báo file conflict trong Source Control ở nhóm **Merge Changes**, và chèn các dấu vào file:

```
<<<<<< HEAD (Current - code CỦA BẠN)
    epochs = 100
=====
    epochs = 50
>>>>>> feature/tuning (Incoming - code nhánh ĐEM VỀ)
```

- **Current change** = phần trên (HEAD) = nhánh bạn đang đứng.
- **Incoming change** = phần dưới = nhánh bạn đang gộp vào.



Hình 1 — Merge Editor 3 khung của VSCode: Current | Incoming | Result

Cách 1 — Dùng nút CodeLens ngay trên file (nhANH NHẤT)

Ngay phía trên khối conflict, VSCode hiện 4 nút chữ xanh — bấm thẳng:

Nút	Kết quả
Accept Current Change	Giữ code của bạn (HEAD), bỏ phần kia
Accept Incoming Change	Lấy code nhánh đem về, bỏ code của bạn
Accept Both Changes	Giữ CẢ HAI (nối tiếp nhau)
Compare Changes	Mở so sánh 2 bên trước khi quyết

Cách 2 — Merge Editor (giống Visual Studio nhất)

1. Trong Source Control, nhóm **Merge Changes**, click file conflict.
2. Bấm nút "**Resolve in Merge Editor**" (góc dưới-phải).
3. Tick chọn dòng ở khung **Current** và/hoặc **Incoming** → chúng chảy xuống khung **Result**.
4. Sửa tay khung Result nếu cần, rồi bấm **Complete Merge**.

Khác biệt với Visual Studio cần nhớ: VSCode không tự động stage & commit sau khi resolve. Bạn phải **tự stage** file (dấu ) rồi **commit** để kết thúc merge.

Hoàn tất bằng Terminal (chắc chắn nhất)

```
# sau khi đã sửa hết dấu <<< == >>> trong file
git add src/train/train_obd.py
git commit          # mở message merge mặc định, chỉ cần lưu

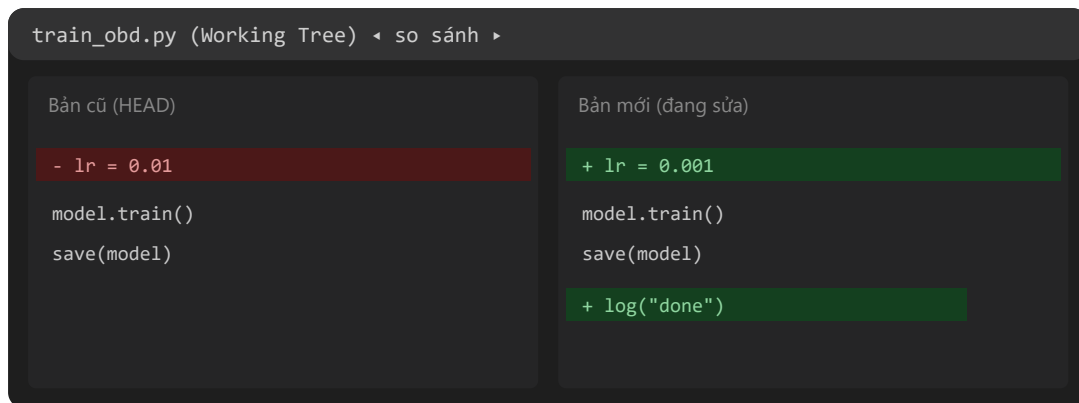
# Lỡ tay muốn HỦY toàn bộ merge, quay lại như trước:
git merge --abort
```

Còn thấy chữ "!" hoặc "Merge Changes" nghĩa là **chưa xong**. Chỉ khi file rời khỏi nhóm Merge Changes và bạn commit thì conflict mới thực sự được giải quyết.

Phần 2 — Xem Diff & Lịch sử một file cụ thể

A. Xem diff các thay đổi CHƯA commit

1. Mở Source Control `Ctrl+Shift+G`.
2. Click vào tên file trong danh sách **Changes**.
3. VSCode mở khung **diff 2 cột**: bên trái = bản cũ (đỏ), bên phải = bản mới (xanh).



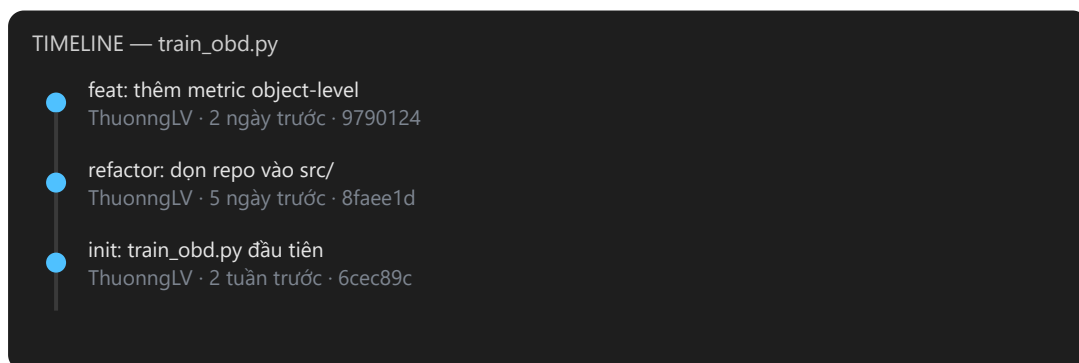
Hình 2 — Khung diff 2 cột: dòng xóa màu đỏ, dòng thêm màu xanh

Bấm icon **hai tờ giấy** góc trên-phải khung diff để đổi qua lại giữa **Side-by-side** (2 cột) và **Inline** (1 cột) — giống Visual Studio.

B. Xem toàn bộ LỊCH SỬ của một file — cần GitLens

VSCode gốc không có "History" cho từng file như VS. Hãy cài extension **GitLens** (miễn phí, phổ biến nhất). Sau khi cài:

- **File History**: chuột phải vào file (trong Explorer hoặc tab) → **"Open File History"**. Cột Timeline hiện mọi commit đụng tới file này.
- **Timeline view**: mở panel Explorer, mục **Timeline** dưới cùng — click từng commit để xem diff của lần đó.
- **Blame (ai sửa dòng nào)**: GitLens hiện chữ mở cuối mỗi dòng: *tên người + thời gian + commit*.



Hình 3 — Timeline/File History của GitLens: mỗi chấm là 1 commit, click để xem diff

C. Bằng Terminal (không cần cài gì)

```
# Lịch sử commit gọn của 1 file
git log --oneline -- src/train/train_obd.py

# Xem file đã đổi gì qua từng commit
git log -p -- src/train/train_obd.py
```

```
# Ai sửa dòng nào (blame)
git blame src/train/train_obd.py
```

Phần 3 — Thiết lập Push / Pull với GitHub

Repo `train` của bạn đã là git repo. Phần này giúp kết nối nó với GitHub và push/pull mượt mà, không bị hỏi mật khẩu liên tục.

Bước 1 — Đăng nhập GitHub trong VSCode (1 lần duy nhất)

1. Click icon **Accounts** (hình người) góc **dưới-trái** VSCode.
2. Chọn **"Sign in with GitHub"** → trình duyệt mở → Authorize.
3. Xong: từ nay push/pull không phải nhập token thủ công nữa.

Đây là cách thay cho việc dán **Personal Access Token** — VSCode giữ đăng nhập giúp bạn, tương tự cách Visual Studio quản lý tài khoản GitHub.

Bước 2 — Khai báo danh tính (nếu chưa)

```
git config --global user.name "ThuongLV"
git config --global user.email "vanthuong2792@gmail.com"
```

Bước 3 — Nối repo local với GitHub

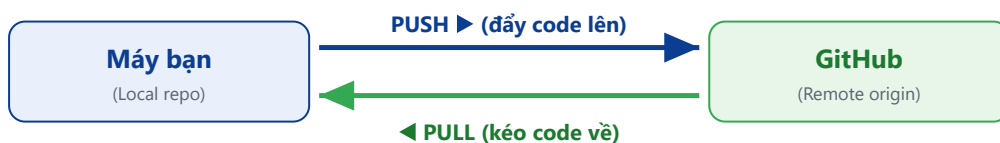
Nếu repo chưa có remote (chưa từng push lên GitHub):

- Cách nhanh: `Ctrl+Shift+P` → **"Publish to GitHub"**. VSCode tự tạo repo trên GitHub và push lên — chọn Private hay Public là xong.
- Hoặc thủ công bằng terminal:

```
git remote add origin https://github.com/<tên-bạn>/train.git
git push -u origin master # -u: lần đầu, gắn nhánh theo dõi
```

Nếu đã có remote, kiểm tra bằng: `git remote -v`

Bước 4 — Vòng lặp Push / Pull hằng ngày



Hình 4 — Push đẩy commit lên GitHub; Pull kéo commit của người khác về máy

Việc	Bảng giao diện	Bảng terminal
Kéo code mới về	Menu ... > Pull	<code>git pull</code>
Đẩy code lên	Menu ... > Push	<code>git push</code>
Vừa pull vừa push	Nút  Sync Changes (status bar)	<code>git pull && git push</code>

↵ master

 1↓ 2↑

↓ = số commit cần Pull về

↑ = số commit cần Push lên · click để Sync

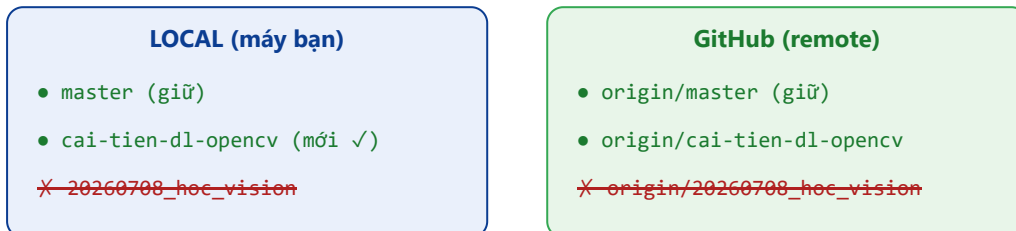
Thói quen tốt: luôn **Pull trước khi bắt đầu code** và **Push ngay sau khi commit**. Làm vậy giảm hẳn nguy cơ conflict ở Phần 1.

Push bị từ chối (rejected)? Nghĩa là trên GitHub có commit mới bạn chưa có. Chạy `git pull` trước (có thể phải giải quyết conflict), rồi `git push` lại.

Phần 4 — Xóa branch cũ & tạo branch mới

Sau khi một nhánh tính năng đã được gộp (merge) vào `master`, bạn nên xóa nó cho gọn. Một branch tồn tại ở **2 nơi** — dưới máy (local) và trên GitHub (remote) — nên muốn "xóa hết" phải xóa cả hai.

Nguyên tắc an toàn: không xóa nhánh bạn đang đứng, và không xóa `master` (nhánh chính). Luôn `git switch master` về nhánh chính trước khi xóa các nhánh khác.



Hình 6 — Xóa nhánh cũ phải làm ở cả 2 nơi: local và GitHub

A. Bằng giao diện VSCode

- **Xóa local:** `Ctrl+Shift+P` → gõ "**Git: Delete Branch**" → chọn nhánh cần xóa.
- **Xóa trên GitHub:** `Ctrl+Shift+P` → "**Git: Push to...**" thường không xóa được; cách chắc chắn là dùng terminal bên dưới (hoặc vào GitHub → tab Branches → thùng rác).
- **Tạo nhánh mới:** click tên nhánh góc **dưới-trái** → "**Create new branch**" → gõ tên.

B. Bằng Terminal (đầy đủ & chắc chắn nhất)

```
# 0. Về master trước cho an toàn
git switch master

# 1. Xóa nhánh cũ DƯỚI MÁY
git branch -d 20260708_hoc_vision # -d: chỉ xóa nếu đã merge (an toàn)
git branch -D 20260708_hoc_vision # -D: xóa ép, kể cả chưa merge

# 2. Xóa nhánh đó TRÊN GITHUB
git push origin --delete 20260708_hoc_vision

# 3. Dọn ref cache cũ (nếu vẫn thấy origin/... đã xóa)
git remote prune origin

# 4. Tạo nhánh mới từ master và chuyển sang
git switch -c cai-tien-dl-opencv

# 5. Đẩy nhánh mới lên GitHub + gắn tracking (-u)
git push -u origin cai-tien-dl-opencv
```

Lệnh	Ý nghĩa
<code>git branch -d X</code>	Xóa local nhánh X (chỉ khi đã merge — bảo vệ khỏi mất code)
<code>git branch -D X</code>	Xóa ép local, kể cả chưa merge (cẩn thận: mất commit chưa gộp)
<code>git push origin --delete X</code>	Xóa nhánh X trên GitHub

<code>git remote prune origin</code>	Dọn các <code>origin/*</code> đã bị xóa ở GitHub nhưng còn hiện dưới máy
<code>git switch -c X</code>	Tạo nhánh mới X và chuyển sang luôn

Lưu ý tên nhánh: git không cho dấu cách trong tên branch. Vì vậy "cải tiến deep learning và opencv" được đặt thành `cai-tien-dl-opencv` (không dấu, dùng gạch nối) để tương thích tốt với mọi công cụ.

Trạng thái repo sau khi dọn: chỉ còn `master` và `cai-tien-dl-opencv` ở cả local lẫn GitHub; nhánh cũ `20260708_hoc_vision` đã bị xóa hoàn toàn. Bạn đang đứng trên nhánh mới, sẵn sàng code.